

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	6
1 Аналитический раздел	7
1.1 Анализ предметной области	7
1.2 Анализ существующих решений	7
1.3 Формулировка требований к базе данных и приложению	8
1.4 Формализация информации, хранимой в базе данных	8
1.5 Выбор методологии проектирования	9
1.6 Ролевая модель	10
1.7 Анализ существующих моделей хранения данных	11
1.7.1 Дореляционные модели	12
1.7.2 Реляционная модель	14
1.7.3 Постреляционные модели	15
1.7.4 Выбор модели	17
2 Конструкторский раздел	18
2.1 Проектирование базы данных	18
2.2 Доказательство соответствия базы данных условиям ЗНФ	22
2.3 Ролевая модель на уровне базы данных	24
2.4 Проектирование способов взаимодействия с базой данных	25
2.5 Разработка функций базы данных	25
3 Технологический раздел	27
3.1 Выбор средств реализации	27
3.2 Проектирование программного обеспечения	28
3.3 Разработка программного обеспечения	30
3.4 Тестирование программного обеспечения	30
4 Исследовательский раздел	33
4.1 Методика и параметры эксперимента	33
4.2 Анализ результатов	34

ЗАКЛЮЧЕНИЕ	37
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	38
ПРИЛОЖЕНИЕ А	40
ПРИЛОЖЕНИЕ Б	41

ВВЕДЕНИЕ

Цель курсовой работы заключается в разработке базы данных для видеохостинг платформы и приложения, осуществляющего к ней доступ.

Для достижения поставленной цели были определены следующие задачи:

- 1) провести анализ предметной области;
- 2) провести анализ существующих решений и сформулировать требования к базе данных и приложению;
- 3) провести анализ существующих моделей хранения данных;
- 4) спроектировать базу данных и ролевую модель на уровне базы данных;
- 5) разработать приложение, обеспечивающее доступ к базе данных;
- 6) описать тестирование разработанного программного обеспечения;
- 7) провести исследование зависимости времени выполнения функции массового уведомления подписчиков канала о новом видео от количества подписчиков при различных стратегиях индексирования.

1 Аналитический раздел

1.1 Анализ предметной области

Предметной областью данного проекта является видеохостинг-платформа, в которой сочетаются две основные стороны работы системы: просмотр контента и его публикация. С одной стороны, пользователь пользуется платформой как зритель: ищет и просматривает интересные видео. С другой стороны, пользователь может выступать как автор, то есть загружать собственные видео в общий доступ.

С точки зрения структуры данных, основные компоненты предметной области включают:

- пользователей, которые взаимодействуют с контентом;
- каналы, выступающие в роли средства передачи контента от автора к зрителям;
- видео, предоставляемые в результате поиска.

Для предметной области также характерны следующие пользовательские взаимодействия:

- подписка на канал, которая служит основанием для уведомлений пользователя о новых публикациях (под уведомлением понимается любой способ информирования пользователя);
- комментарии пользователей к видео;
- оценивание видео и комментариев: положительная реакция «лайк» или отрицательная — «дизлайк»;
- объединение набора видеозаписей в единую структуру «плейлист».

Таким образом, требуется хранить не только сведения о самих видеозаписях, но и связи между пользователями, каналами и действиями, которые выполняются в системе.

1.2 Анализ существующих решений

Существуют готовые решения, предоставляющие пользователям возможность просмотра и публикации видеоконтента. Рассмотрены три платформы:

- VK Video [1];
- RuTube [2];

— Дзен Видео [3].

Анализ проведён в сравнительном формате по выделенным функциональным критериям. Результаты анализа представлены в таблице 1.1.

Таблица 1.1 — Сравнение существующих решений предметной области

Критерий	VK Video	RuTube	Дзен Видео
Формирование плейлистов	Да	Да	Нет
Отрицательная оценка комментариев	Нет	Да	Нет
Отрицательная оценка видео	Нет	Нет	Да
Сортировка видео канала по просмотрам	Да	Нет	Да
Сортировка видео канала по оценкам	Нет	Нет	Нет

Таким образом, проанализированные решения не представляют полноценной возможности оценивать контент и сортировать список видеозаписей канала.

1.3 Формулировка требований к базе данных и приложению

Необходимо разработать базу данных, которая будет хранить видеофайлы, метаданные и информацию о следующих взаимодействиях пользователей с платформой:

- просмотр, оценивание и комментирование видеозаписей;
- оценивание комментариев;
- создание канала и публикация собственных видео;
- объединение видео своего канала в плейлисты;
- отправка и получение уведомлений о новых видео.

Требуется разработать приложение, предоставляющее возможность осуществлять вышеупомянутые взаимодействия, выполнять поиск видео и получить список видеозаписей какого-либо канала.

1.4 Формализация информации, хранимой в базе данных

В соответствии со сформулированными требованиями определены сущности, которые будут храниться в базе данных. Информация о сущностях и их атрибутах представлена в таблице 1.2.

Таблица 1.2 — Сущности и их атрибуты

Сущность	Атрибуты
Роль	название, описание
Пользователь	псевдоним, роль, электронная почта, пароль, уведомления вкл./выкл.
Канал	название, владелец, описание, дата регистрации
Видео	канал, название, описание, дата публикации, видеофайл
Плейлист	канал, название, описание, дата создания
Просмотр	видео, пользователь, дата просмотра
Комментарий	видео, автор, текст, дата написания

Сущности и их связи представлены на ER-диаграмме в нотации Чена на рисунке 1.1.

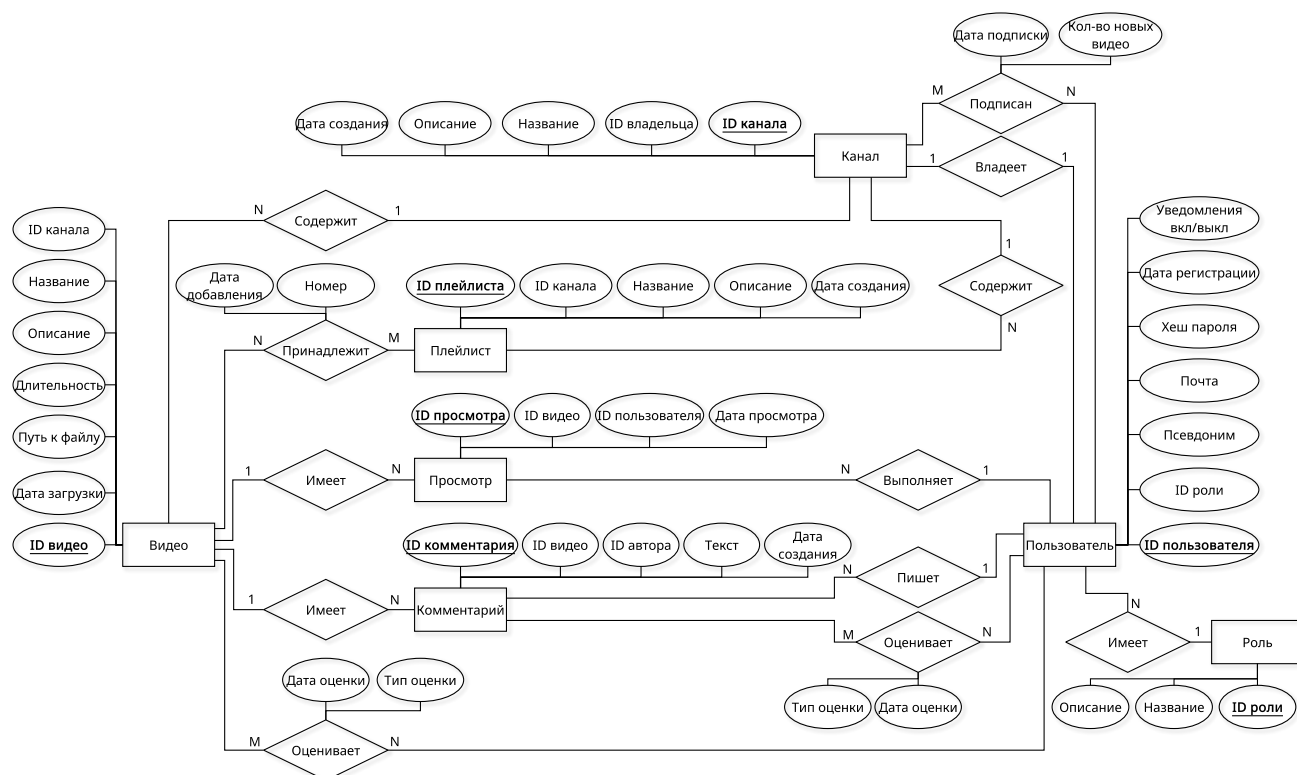


Рисунок 1.1 — ER-диаграмма предметной области

1.5 Выбор методологии проектирования

При проектировании информационных систем используются структурный и объектно-ориентированный подходы. В рамках данной работы выбран

объектно-ориентированный подход, так как он позволяет описать пользователей, их сценарии взаимодействия и связи сущностей через диаграммы. В качестве ключевых используются диаграмма вариантов использования и ER-диаграмма сущностей.

1.6 Ролевая модель

Предметная область подразумевает введение ролевой модели из четырёх ролей:

- гость, который является неавторизованным пользователем;
- пользователь, который просматривает, публикует и взаимодействует с видео;
- модератор, который может удалять пользовательский контент;
- администратор, который может блокировать пользователей и менять их роли.

Пользователи приложения формализованы через перечисленные роли, каждая из которых соответствует набору доступных сценариев взаимодействия с системой. Диаграмма вариантов использования представлена на рисунке 1.2.

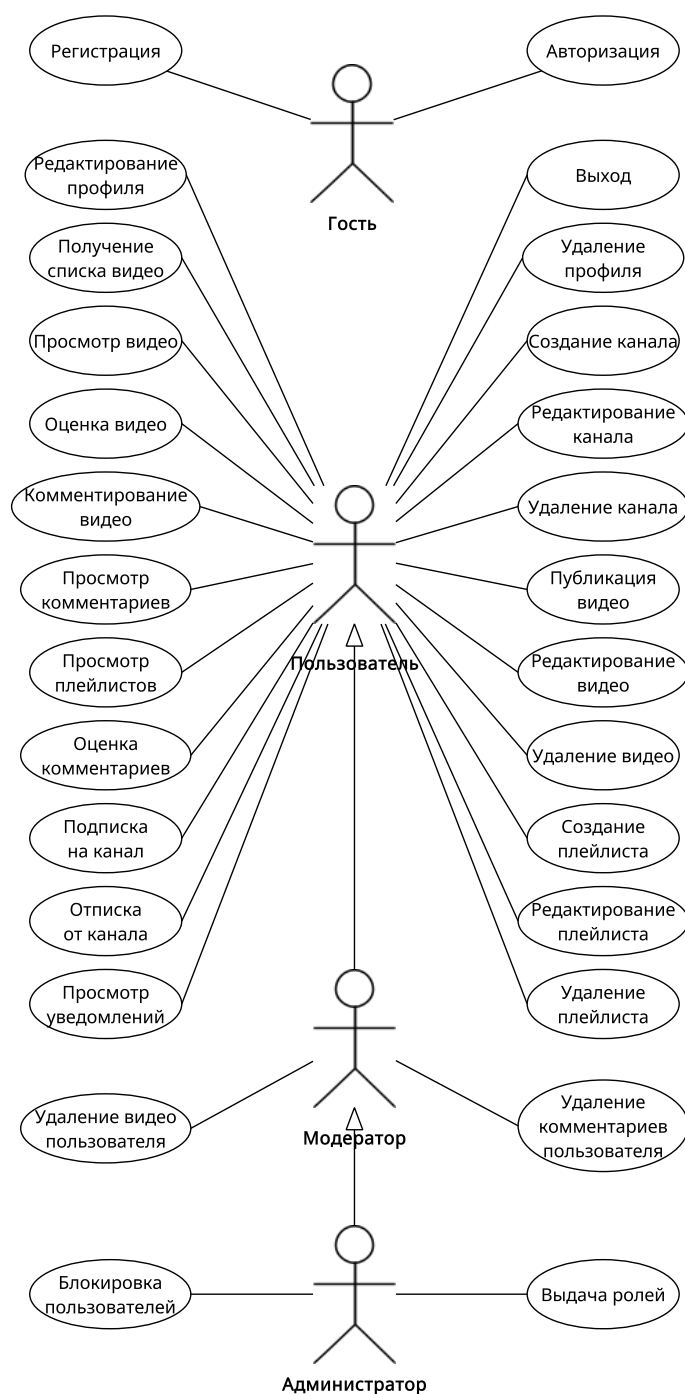


Рисунок 1.2 — Диаграмма вариантов использования

1.7 Анализ существующих моделей хранения данных

База данных представляет собой интегрированное хранилище, которое содержит не только сами данные, но и информацию о связях между этими данными, что позволяет пользователю или прикладной программе манипулировать данными, не вникая в детали их физического размещения на носителе [4].

Под моделью хранения данных понимается способ физической и логиче-

ской организации данных, который определяет, где и как данные фиксируются, как они структурированы и какие взаимосвязи между ними могут быть установлены для обеспечения работы бизнеса [4]. Рассматриваются три основных типа моделей организации данных [4]:

- дореляционные модели;
- реляционная модель;
- постреляционные модели.

1.7.1 Дореляционные модели

Выделяют следующие дореляционные модели хранения данных:

- модель на основе инвертированных списков;
- иерархическую модель;
- сетевую модель.

Модель на основе инвертированных списков

Ключевая идея модели заключается в изменении ролей записей и атрибутов. Вместо хранения списка атрибутов для каждой записи, для каждого значения атрибута строится список идентификаторов записей, которые содержат это значение [5]. Это позволяет находить записи, удовлетворяющие условию, быстрее, чем если бы проверялась каждая запись [5].

Файл с инвертированными списками обычно существует вместе с основным файлом данных, предоставляя избыточную, дублирующую информацию для ускорения поиска [5]. Пример организации модели представлен на рисунке 1.3.



Рисунок 1.3 — Пример организации модели на основе инвертированных списков

Помимо дублирования данных, среди ограничений также выделяют сложность поддержки при частых обновлениях файла данных: данная модель не оптимизирована для оперативной синхронизации в реальном времени [5].

Иерархическая модель

Модель организована в виде набора иерархических структур данных (деревьев), где доступ к данным возможен только через корневой узел иерархии [5]. В иерархической модели поддерживается только связь типа «один-ко-многим» (1:N), причём ни одна дочерняя сущность не может иметь более одной родительской сущности [5]. Схема иерархической модели представлена на примере на рисунке 1.4.

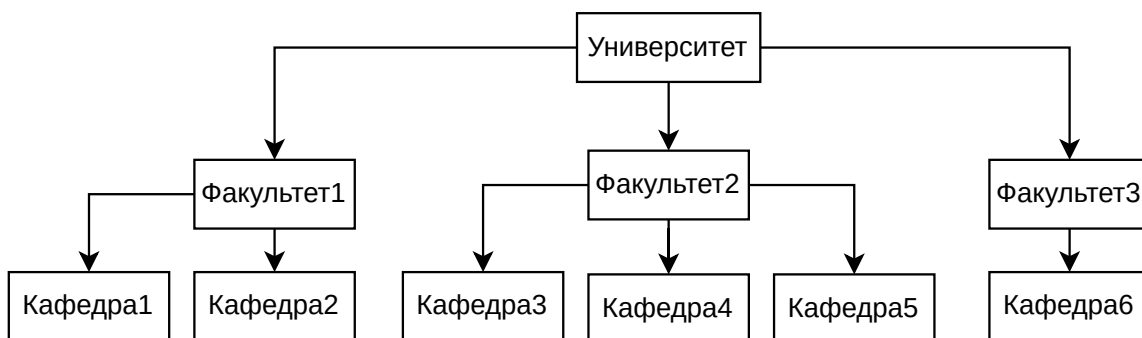


Рисунок 1.4 — Пример организации данных на основе иерархической модели

Иерархическая модель обеспечивает высокую скорость обработки в порядке обхода дерева, но делает произвольный доступ крайне медленным [5]. Кроме того, из-за запрета на множественное наследование возникают дублирования данных [5].

Сетевая модель

Сетевая модель появилась вслед за иерархической с целью устранить недостатки последней. Она снимает ограничение на множественное наследование, позволяет напрямую реализовывать отношения «многие-ко-многим» (M:N) [5]. Схема сетевой модели представлена на рисунке 1.5.

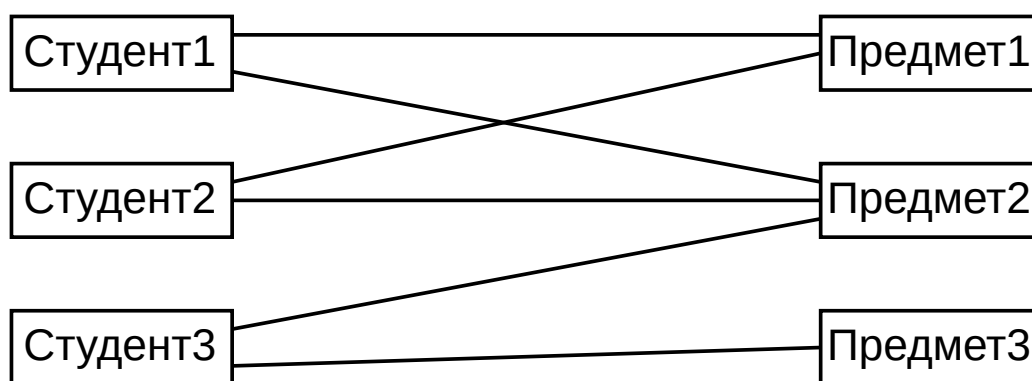


Рисунок 1.5 — Пример организации модели на основе инвертированных списков

Если связь M:N имеет собственные атрибуты, в такой структуре их негде хранить, причём при добавлении третьей сущности для решения этой проблемы связи могут стать двусмысленными [5].

1.7.2 Реляционная модель

Реляционная модель представляет собой способ организации данных, при котором вся информация представляется в виде двумерных отношений, то есть таблиц, состоящих из строк и столбцов. Для строки отношение требует единственного значения в каждой ячейке, отсутствия дубликатов и наличия первичного ключа, который однозначно идентифицирует запись. Взаимодействие с реляционными базами данных выполняется с помощью стандартного языка обработки данных SQL. [5] Пример представления информации в реляционной модели представлен на рисунке 1.6.

№	Фамилия	Курс	Факультет	Телефон
1	Иванов	2	ИУ	213-1231-1123
2	Попов	1	ИУ	222-444-111
3	Петров	3	ФН	00-00-00
...	...	...	...	...

Рисунок 1.6 — Пример таблицы реляционной базы данных

Связи между сущностями в реляционной модели задаются через первичные и внешние ключи, а их корректность контролируется механизмом ссылочной целостности, который требует, чтобы каждое значение внешнего ключа соответствовало существующей записи в связанной таблице. Это позволяет хранить связанные данные в разных таблицах без потери логики предметной области. Связь вида M:N в такой модели реализуется через промежуточные таблицы, которые могут хранить собственные атрибуты. [5]

1.7.3 Постреляционные модели

Тенденции в глобализации производства формулируют новые требования к хранилищам данных [6]:

- создание моделей данных, не требующих строго фиксированной структуры;
- хранение в объекте базы данных не только самих данных, но и способов их обработки;
- учёт взрывного роста объёмов хранимых данных.

Так, были разработаны следующие основные постреляционные модели хранения данных [6]:

- модель key-value;
- модель объектного хранилища;
- объектно-ориентированная модель.

Модель key-value

Ключевая идея key-value модели заключается в хранении данных в виде пар, состоящих из уникального идентификатора и ассоциированного с ним значения. Для каждого ключа в хранилище предусмотрено ровно одно значение, а доступ к данным осуществляется путём указания ключа с помощью простых команд. В отличие от реляционных моделей, key-value хранилища не поддерживают ни вложенности, ни ссылочной целостности. Key-value модели характеризуются полным отсутствием фиксированной схемы: данные могут сохраняться в произвольных форматах без предварительного определения таблиц и атрибутов. [7]

Объектное хранилище

В объектном хранилище объект является логической коллекцией байтов переменной длины, которая может хранить целые структуры данных: файлы, записи баз данных, изображения или мультимедийный контент. Каждый объект в такой модели состоит из трёх компонентов: данных, доступных пользователю атрибутов и метаданных, управляемых самим устройством. Устройство объектного хранения самостоятельно управляет размещением данных на физическом носителе, включая распределение свободного пространства. [8]

Объектно-ориентированная модель

Объектно-ориентированные базы данных обеспечивают создание долговременно хранящихся объектов и средства для поиска и управления этими объектами [6].

Объектная специфика данных реализуется следующим образом [6]:

- объект имеет уникальный идентификатор и наборы изменяемых свойств;
- модель содержит исходный набор встроенных объектных и структурных типов, из которых возможно создавать новые типы и моделировать схемы данных;
- модель состоит из наборов объектов, каждый из которых является экземпляром конкретного типа.

1.7.4 Выбор модели

Сравнительный анализ моделей хранения данных проведён на основе следующих критериев:

- К1 — поддержка связей типа 1:1, 1:N, M:N;
- К2 — поддержка ACID;
- К3 — хранение файлов;
- К4 — временная сложность запроса $O(1)$ (для кеширования);
- К5 — нормализация (отсутствие избыточности).

Результаты анализа представлены в таблице 1.3.

Таблица 1.3 — Сравнение моделей хранения данных

Модель хранения данных	К1	К2	К3	К4	К5
Инвертированные списки	—	—	—	+	—
Иерархическая модель	—	—	—	—	—
Сетевая модель	+	+	—	—	—
Реляционная модель	+	+	—	—	+
Модель key-value	—	—	—	+	—
Объектно-ориентированная модель	+	+	—	—	—
Объектное хранилище	—	—	+	+	—

Для хранения метаданных о видео и информации о связях, возникающих в системе, выбрана классическая реляционная модель хранения. Для хранения видео выбрано объектное хранилище.

Выводы по разделу

В аналитическом разделе была проанализирована предметная область видеохостинга, выделены основные сущности и взаимодействия. На основе анализа существующих решений сформулированы требования к разрабатываемой базе данных и приложению. Формализована информация о сущностях и их атрибутах, построена ER-диаграмма. Разработана ролевая модель и диаграмма вариантов использования. В результате анализа моделей хранения данных для метаданных и связей выбрана реляционная модель, для хранения видеофайлов — объектное хранилище.

2 Конструкторский раздел

2.1 Проектирование базы данных

Согласно ER-диаграмме предметной области, представленной на рисунке 1.1, проектируемая база данных должна содержать следующие таблицы (с указанными ограничениями целостности):

- *roles* с информацией о ролях:
 - *id* — идентификатор роли (первичный ключ, целочисленный тип);
 - *name* — название роли (строковый тип, непустое значение);
 - *is_default* — флаг, определяющий роль, присваиваемую новым пользователям по умолчанию (логический тип, по умолчанию *FALSE*).
- *users* с информацией о пользователях приложения:
 - *id* — идентификатор пользователя (первичный ключ, целочисленный тип);
 - *role_id* — роль пользователя (внешний ключ на таблицу *roles*, целочисленный тип, непустое значение);
 - *username* — псевдоним пользователя (строковый тип, уникальное и непустое значение);
 - *email* — адрес электронной почты (строковый тип, уникальное и непустое значение);
 - *password_hash* — хеш пароля пользователя (строковый тип, непустое значение);
 - *notifications_enabled* — флаг включения/выключения уведомлений (логический тип, непустое значение, по умолчанию *TRUE*);
 - *is_active* — флаг блокировки пользователя (логический тип, непустое значение, по умолчанию *TRUE*);
 - *created_at* — дата и время регистрации (тип даты и времени, непустое значение, по умолчанию текущее время);
 - *updated_at* — дата и время последнего обновления профиля (тип даты и времени, непустое значение, по умолчанию текущее время).
- *channels* с информацией о каналах, принадлежащих пользователям:
 - *id* — идентификатор канала (первичный ключ, целочисленный тип);

- *user_id* — владелец канала (внешний ключ на таблицу *users*, целочисленный тип, непустое значение);
 - *name* — название канала (строковый тип, уникальное и непустое значение);
 - *description* — текстовое описание канала (строковый тип);
 - *created_at* — дата и время создания канала (тип даты и времени, непустое значение, по умолчанию текущее время).
- *videos* с метаданными видеозаписей:
- *id* — идентификатор видео (первичный ключ, целочисленное значение);
 - *channel_id* — канал-владелец видео (внешний ключ на таблицу *channels*, целочисленный тип, непустое значение);
 - *title* — название видео (строковый тип, непустое значение);
 - *description* — текстовое описание видео (строковый тип);
 - *filepath* — путь к файлу с видеозаписью (строковый тип, непустое значение);
 - *created_at* — дата и время публикации видео (тип даты и времени, непустое значение, по умолчанию текущее время).
- *playlists* с информацией о плейлистах, созданных на канале:
- *id* — идентификатор плейлиста (первичный ключ, целочисленный тип);
 - *channel_id* — канал-владелец плейлиста (внешний ключ на таблицу *channels*, целочисленный тип, непустое значение);
 - *name* — название плейлиста (строковый тип, непустое значение);
 - *description* — текстовое описание плейлиста (строковый тип);
 - *created_at* — дата и время создания плейлиста (тип даты и времени, непустое значение, по умолчанию текущее время).
- *playlist_items* для связи плейлистов с видео (формализация связи «многие-ко-многим» между *playlists* и *videos*):
- *playlist_id* — идентификатор плейлиста (часть составного первичного ключа, внешний ключ на таблицу *playlists*, целочисленный тип);
 - *video_id* — идентификатор видео (часть составного первичного

- ключа, внешний ключ на таблицу *videos*, целочисленный тип);
 - *number* — порядковый номер видео в плейлисте (целочисленный тип);
 - *added_at* — дата и время добавления видео в плейлист (тип даты и времени, непустое значение, по умолчанию текущее время).
- *viewings* для хранения истории просмотров видео пользователями:
 - *id* — идентификатор просмотра (первичный ключ, целочисленный тип);
 - *user_id* — идентификатор пользователя (внешний ключ на таблицу *users*, целочисленный тип, непустое значение);
 - *video_id* — идентификатор видео (внешний ключ на таблицу *videos*, целочисленный тип, непустое значение);
 - *watched_at* — дата и время просмотра (тип даты и времени, непустое значение, по умолчанию текущее время).
- *comments* для хранения комментариев пользователей к видео:
 - *id* — идентификатор комментария (первичный ключ, целочисленный тип);
 - *user_id* — автор комментария (внешний ключ на таблицу *users*, целочисленный тип, непустое значение);
 - *video_id* — идентификатор видео (внешний ключ на таблицу *videos*, целочисленный тип, непустое значение);
 - *content* — текстовое содержание комментария (строковый тип, непустое значение);
 - *created_at* — дата и время написания комментария (тип даты и времени, непустое значение, по умолчанию текущее время).
- *video_ratings* для хранения оценок видео (формализация связи «многие-ко-многим» между *users* и *videos*):
 - *user_id* — идентификатор пользователя (часть составного первичного ключа, внешний ключ на таблицу *users*, целочисленный тип);
 - *video_id* — идентификатор видео (часть составного первичного ключа. внешний ключ на таблицу *videos*, целочисленный тип);
 - *liked* — флаг лайк/дизлайк (логический тип, непустое значение);

- *rated_at* — дата и время выставления оценки (тип даты и времени, непустое значение, по умолчанию текущее время).
- *comment_ratings* для хранения оценок комментариев (формализация связи «многие-ко-многим» между *users* и *comments*):
 - *user_id* — идентификатор пользователя (часть составного первичного ключа, внешний ключ на таблицу *users*, целочисленный тип);
 - *comment_id* — идентификатор комментария (часть составного первичного ключа, внешний ключ на таблицу *comments*, целочисленный тип);
 - *liked* — флаг лайк/дизлайк (логический тип, непустое значение);
 - *rated_at* — дата и время выставления оценки (тип даты и времени, непустое значение, по умолчанию текущее время).
- *subscriptions* для хранения информации о подписках пользователей на каналы (формализация связи «многие-ко-многим» между *users* и *channels*):
 - *user_id* — идентификатор пользователя (часть составного первичного ключа, внешний ключ на таблицу *users*, целочисленный тип);
 - *channel_id* — идентификатор канала (часть составного первичного ключа, внешний ключ на таблицу *channels*, целочисленный тип);
 - *new_videos_count* — счётчик новых видео для уведомления (целочисленный тип, по умолчанию 0);
 - *subscribed_at* — дата и время подписки (тип даты и времени, непустое значение, по умолчанию текущее время).

Диаграмма проектируемой базы данных представлена на рисунке 2.1.



Рисунок 2.1 — Диаграмма базы данных

2.2 Доказательство соответствия базы данных условиям 3НФ

Таблица находится в 1НФ, если выполняются следующие условия:

- 1) каждая строка таблицы уникальна;
- 2) каждый столбец содержит только одно значение.

Таблица находится в 2НФ, если выполняются следующие условия:

- 1) таблица находится в 1НФ;
- 2) любой неключевой атрибут должен зависеть от всех атрибутов ключа.

Таблица находится в 3НФ, если выполняются следующие условия:

- 1) таблица находится в 2НФ;
- 2) каждый не ключевой атрибут нетранзитивно зависит от первичного ключа.

Все таблицы находятся в 1НФ, поскольку в каждой таблице есть первичный ключ, обеспечивающий уникальность строки, и каждый атрибут выделен в отдельный столбец, что обеспечивает простоту содержимого столбца.

Для дальнейшего доказательства необходимо рассмотреть функциональные зависимости каждой таблицы:

$$\begin{aligned}
\mathcal{F}_{\text{roles}} &= \left\{ \begin{array}{l} \text{id} \rightarrow \{\text{name}, \text{is_default}\}, \\ \text{name} \rightarrow \{\text{id}, \text{is_default}\} \end{array} \right\}, \\
\mathcal{F}_{\text{users}} &= \left\{ \begin{array}{l} \text{id} \rightarrow \left\{ \begin{array}{l} \text{role_id}, \text{username}, \text{email}, \\ \text{password_hash}, \text{notifications_enabled}, \\ \text{is_active}, \text{created_at}, \text{updated_at} \end{array} \right\}, \\ \text{username} \rightarrow \left\{ \begin{array}{l} \text{id}, \text{role_id}, \text{email}, \\ \text{password_hash}, \text{notifications_enabled}, \\ \text{is_active}, \text{created_at}, \text{updated_at} \end{array} \right\}, \\ \text{email} \rightarrow \left\{ \begin{array}{l} \text{id}, \text{role_id}, \text{username}, \\ \text{password_hash}, \text{notifications_enabled}, \\ \text{is_active}, \text{created_at}, \text{updated_at} \end{array} \right\} \end{array} \right\}, \\
\mathcal{F}_{\text{channels}} &= \left\{ \begin{array}{l} \text{id} \rightarrow \{\text{user_id}, \text{name}, \text{description}, \text{created_at}\}, \\ \text{name} \rightarrow \{\text{id}, \text{user_id}, \text{description}, \text{created_at}\} \end{array} \right\}, \\
\mathcal{F}_{\text{videos}} &= \left\{ \text{id} \rightarrow \left\{ \begin{array}{l} \text{channel_id}, \text{title}, \text{description}, \\ \text{filepath}, \text{created_at} \end{array} \right\} \right\}, \\
\mathcal{F}_{\text{playlists}} &= \left\{ \text{id} \rightarrow \{\text{channel_id}, \text{name}, \text{description}, \text{created_at}\} \right\}, \\
\mathcal{F}_{\text{viewings}} &= \left\{ \text{id} \rightarrow \{\text{user_id}, \text{video_id}, \text{watched_at}\} \right\}, \\
\mathcal{F}_{\text{comments}} &= \left\{ \text{id} \rightarrow \{\text{user_id}, \text{video_id}, \text{content}, \text{created_at}\} \right\}, \\
\mathcal{F}_{\text{subscriptions}} &= \left\{ \left\{ \text{user_id}, \text{channel_id} \right\} \rightarrow \left\{ \begin{array}{l} \text{new_videos_count}, \\ \text{subscribed_at} \end{array} \right\} \right\}, \\
\mathcal{F}_{\text{playlist_items}} &= \left\{ \left\{ \text{playlist_id}, \text{video_id} \right\} \rightarrow \{\text{number}, \text{added_at}\} \right\}, \\
\mathcal{F}_{\text{video_ratings}} &= \left\{ \left\{ \text{user_id}, \text{video_id} \right\} \rightarrow \{\text{liked}, \text{rated_at}\} \right\},
\end{aligned}$$

$$\mathcal{F}_{\text{comment_ratings}} = \left\{ \{ \text{user_id}, \text{comment_id} \} \rightarrow \{ \text{liked}, \text{rated_at} \} \right\}.$$

Во всех функциональных зависимостях в левой части присутствуют только потенциальные ключи. Следовательно, каждый неключевой атрибут нетранзитивно зависит от всех атрибутов первичных ключей. Таким образом, все таблицы находятся в 3НФ.

2.3 Ролевая модель на уровне базы данных

В результате анализа предметной области была введена ролевая модель из следующих четырёх ролей:

- гость;
- пользователь;
- модератор;
- администратор.

На уровне базы данных ролевая модель отражается через привилегии, то есть набор разрешённых операций для каждой роли на каждой таблице. Однако в данном случае требуется более тонкое управление (например, пользователь может изменять в таблице *users* только данные о себе), что должно обрабатываться на уровне приложения.

В соответствии с диаграммой вариантов использования, представленной на рисунке 1.2, определены следующие привилегии.

Привилегии гостя

Гостю доступна только регистрация и авторизация, то есть операции *INSERT*, *SELECT* над таблицей *users*.

Привилегии пользователя

Пользователь может выполнять следующие операции:

- *SELECT* над таблицей *roles*;
- *SELECT*, *UPDATE* над таблицей *users*;
- *INSERT*, *SELECT*, *UPDATE*, *DELETE* над таблицей *channels*;
- *INSERT*, *SELECT*, *UPDATE*, *DELETE* над таблицей *videos*;
- *INSERT*, *SELECT*, *UPDATE*, *DELETE* над таблицей *playlists*;
- *INSERT*, *SELECT* над таблицей *viewings*;

- *INSERT, SELECT, UPDATE, DELETE* над таблицей *comments*;
- *INSERT, SELECT, DELETE* над таблицей *subscriptions*;
- *INSERT, SELECT, UPDATE, DELETE* над таблицей *playlists_items*;
- *INSERT, SELECT, UPDATE, DELETE* над таблицей *video_ratings*;
- *INSERT, SELECT, UPDATE, DELETE* над таблицей *comment_ratings*.

При этом операции *UPDATE, DELETE* пользователь может выполнять только над своими записями в таблице. Кроме того, пользователю недоступно изменение атрибута *is_active* в таблице *users*.

Привилегии модератора

Модератор обладает всеми привилегиями пользователя, но при этом может также выполнять операции *UPDATE, DELETE* над всеми записями в таблицах *videos, comments*.

Привилегии администратора

Администратор может выполнять все операции над всеми таблицами, в том числе изменять атрибут *is_active* в таблице *users*.

2.4 Проектирование способов взаимодействия с базой данных

Взаимодействие приложения с базой данных организовано через REST API, где каждый сценарий пользователя соответствует набору операций *SELECT, INSERT, UPDATE, DELETE*. На уровне базы данных предусмотрены операции для получения списков видео, публикации видео, добавления комментариев и управления подписками. Часть логики уведомления подписчиков вынесена в хранимую процедуру на стороне СУБД.

2.5 Разработка функций базы данных

Схема алгоритма работы функции уведомления подписчиков канала о новом видео представлена на рисунке 2.2.

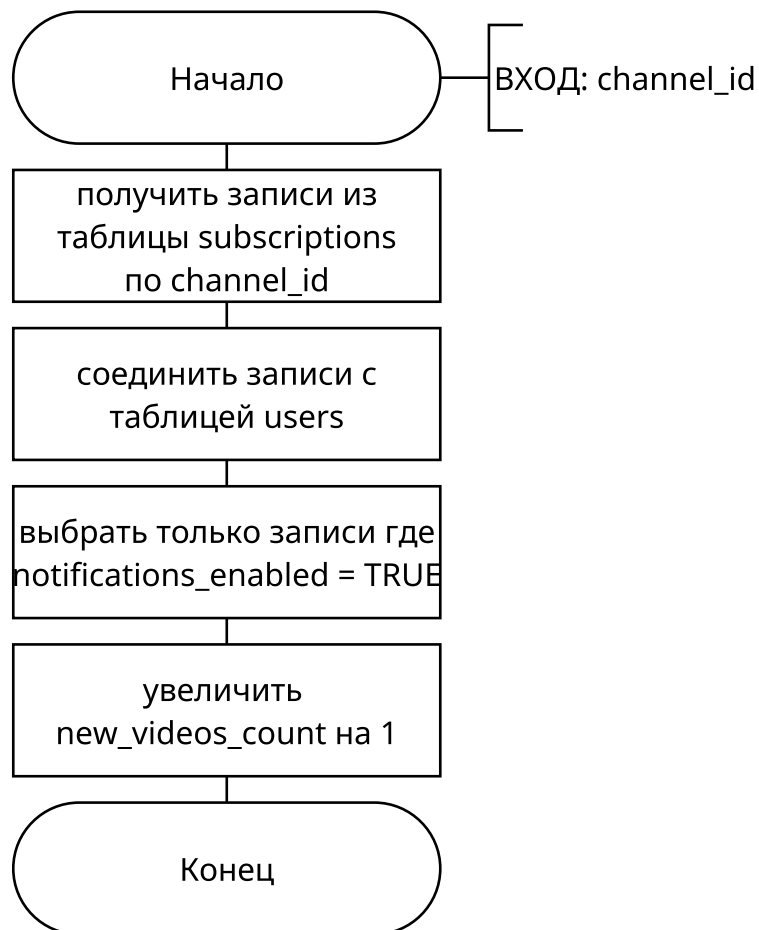


Рисунок 2.2 — Схема алгоритма работы функции уведомления подписчиков канала о новом видео

Выводы по разделу

В конструкторском разделе спроектирована реляционная база данных, соответствующая 3НФ. Разработанная ранее ролевая модель была спроектирована на уровне базы данных: для каждой роли определены привилегии на операции с таблицами. Спроектирована функция уведомления подписчиков канала о новом видео.

3 Технологический раздел

3.1 Выбор средств реализации

Выбор СУБД

Проведён сравнительный анализ следующих систем управления реляционными базами данных: MySQL [9], PostgreSQL [10], Oracle Database [11], Microsoft SQL Server [12] и SQLite [13].

Критерии сравнения:

- К1 — бесплатный доступ;
- К2 — поддержка транзакций;
- К3 — поддержка ролевой модели;

Результаты анализа представлены в таблице 3.1.

Таблица 3.1 — Сравнение СУБД

СУБД	К1	К2	К3
MySQL	+	+	+
PostgreSQL	+	+	+
Oracle Database	—	+	+
Microsoft SQL Server	—	+	+
SQLite	+	—	—

В результате анализа для разработки выбрана СУБД PostgreSQL, поскольку она соответствует сформулированным критериям.

Выбор объектного хранилища

Проведён сравнительный анализ следующих объектных хранилищ: Amazon S3 [14], Google Cloud Storage [15], MinIO [16].

Критерии сравнения:

- бесплатный доступ;
- минимальное потребление ОЗУ;
- модель развёртывания.

Результаты анализа представлены в таблице 3.2.

Таблица 3.2 — Сравнение объектных хранилищ

Объектное хранилище	K1	K2	K3
Amazon S3	—	—	облачная
Google Cloud Storage	—	—	облачная
MinIO	+	+	локальная

В результате анализа для разработки выбрано объектное хранилище MinIO.

Выбор средств реализации программного обеспечения

Для реализации приложения выбран язык программирования Go [17], поскольку он ориентирован на разработку серверных приложений и обладает набором пакетов для работы с REST API, базами данных и хранилищами. Разработка будет осуществляться в среде разработки JetBrains GoLand [18], которая обеспечивает инструменты для написания, отладки и тестирования кода на языке Go.

3.2 Проектирование программного обеспечения

При моделировании архитектуры программной системы использован метод графического моделирования C4. На уровне контейнеров система представлена как связка Web SPA, Backend, базы данных PostgreSQL, кеша Redis и объектного хранилища MinIO. Web SPA взаимодействует с Backend по REST API для операций. Backend хранит метаданные и бизнес-сущности в PostgreSQL, а загрузка/получение видео выполняется через MinIO с использованием presigned URL, выдаваемых Backend. Backend использует Redis для кеширования статистики, счётчиков и refresh-сессий, чтобы снизить нагрузку на БД. Роли пользователей (гость, пользователь, модератор, администратор) отражают сценарии доступа в интерфейсе.

Схема связи компонентов представлена на рисунке 3.1.

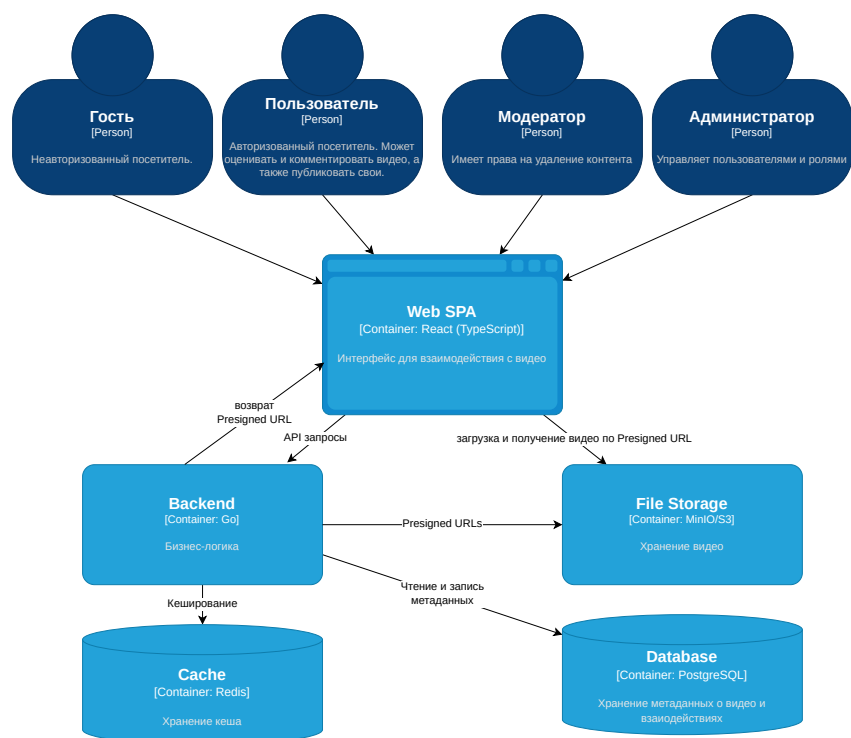


Рисунок 3.1 — Уровень контейнеров архитектуры системы

Для уточнения взаимодействия компонентов приведена диаграмма последовательности процесса загрузки видео. Диаграмма показывает шаги обмена между Web SPA, Backend, PostgreSQL и MinIO, фиксирует порядок вызовов и точки ответственности. Диаграмма последовательности представлена на рисунке 3.2.

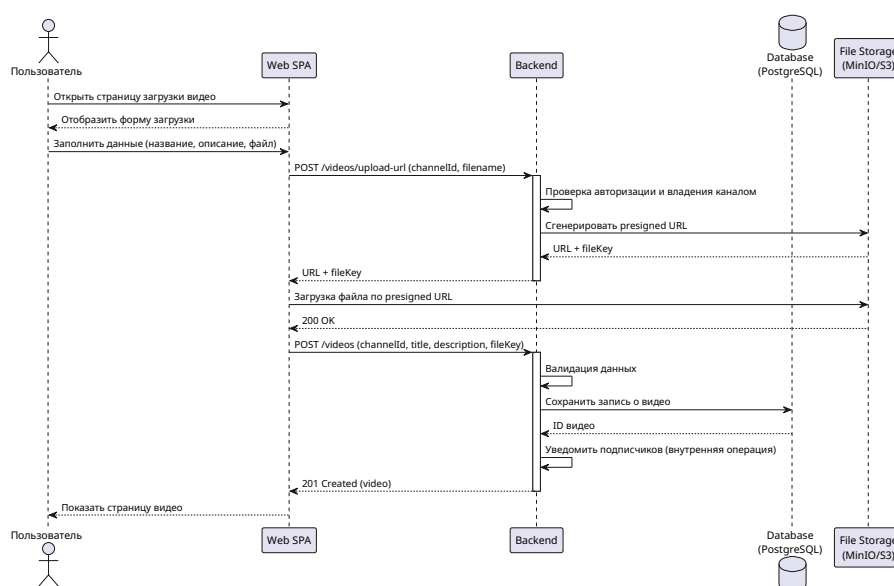


Рисунок 3.2 — Диаграмма последовательности процесса загрузки видео

3.3 Разработка программного обеспечения

Разработка базы данных и приложения доступа

База данных реализована в PostgreSQL с использованием миграций, содержащих создание таблиц, и ограничений целостности. Приложение доступа реализовано в виде REST API, обеспечивающего операции регистрации, публикации видео, управления подписками и взаимодействия с контентом. Для проверки требований сформирован тестовый набор данных не менее 1000 записей для каждой таблицы.

Сценарий создания базы данных приведён в листинге Б.1. На стороне СУБД реализована хранимая функция уведомления подписчиков канала о новом видео, которая выполняет обновление счётчиков новых видео только для подписчиков с включёнными уведомлениями. Реализация функции представлена в листинге Б.2.

Внешнее кэширование с использованием Redis

В качестве InMemory СУБД используется Redis для кэширования счётчиков и агрегированной статистики. Кэш содержит значения просмотров, лайков и дизлайков по видео, а также счётчики новых уведомлений, что снижает нагрузку на базу данных. При записи новых событий происходит инвалидация и обновление кеша при следующем запросе.

3.4 Тестирование программного обеспечения

Проверка корректности работы выполнялась на двух уровнях: модульном и интеграционном. Модульные тесты применялись для сервисного слоя, использовались фиктивные реализации внешних зависимостей, что позволило изолировать бизнес-логику. Интеграционные тесты применялись для слоя репозитория PostgreSQL. Хранимая процедура PostgreSQL для уведомления подписчиков о новом видео также протестирована интеграционными тестами.

Классы эквивалентности для тестирования хранимой процедуры для уведомления подписчиков представлены в таблице 3.3.

Таблица 3.3 — Классы эквивалентности для хранимой процедуры уведомления подписчиков

Параметр	Класс эквивалентности	Ожидаемый результат
channel_id	Канал существует, есть подписчики с включёнными уведомлениями	Счётчики новых видео увеличены
channel_id	Канал существует, подписчиков нет	Изменений нет
channel_id	Канал существует, подписчики есть, но уведомления отключены	Изменений нет
channel_id	Канал не существует	Ошибка/отсутствие изменений
video_id	Видео существует и принадлежит каналу	Уведомление формируется
video_id	Видео не существует или не принадлежит каналу	Ошибка/отсутствие изменений

Примеры тестов, используемых при проверке функциональности, представлены в таблице 3.4.

Таблица 3.4 — Примеры тестов

№ теста	Описание теста	Ожидаемый результат
1	Регистрация пользователя: проверка создания пользователя с валидными данными и назначением роли по умолчанию.	Пользователь создан, роль назначена.
2	Создание канала: проверка, что канал создаётся только при отсутствии канала у пользователя и уникальном имени.	Канал создан, возвращён идентификатор.
3	Уведомление подписчиков: проверка увеличения счётчика новых видео для подписчиков с включёнными уведомлениями.	Счётчик новых видео увеличен.

Все тесты пройдены успешно.

Выводы по разделу

В технологическом разделе обоснован выбор СУБД PostgreSQL, объектного хранилища MinIO и языка Go. Описана архитектура системы на уровне контейнеров и взаимодействие компонентов. Приведён подход к тестированию и набор тестовых примеров.

4 Исследовательский раздел

В данном разделе приведено исследование влияния стратегий индексирования на время выполнения функции `notify_subscribers_about_new_video`. Измерения выполнены для разных масштабов данных и долей подписчиков целевого канала, чтобы понять, какие индексы дают наибольший эффект при росте нагрузки.

Технические характеристики ЭВМ

Замеры проводились на устройстве со следующими характеристиками:

- операционная система — Microsoft Windows 11 Pro (64-разрядная);
- процессор — AMD Ryzen 5 3600X, 6 физических ядер, 12 логических ядер, 3.79 ГГц;
- оперативная память — 16 Гбайт.

4.1 Методика и параметры эксперимента

Для каждого набора параметров база данных пересоздавалась и заполнялась данными. Количество пользователей/подписок варьировалось в диапазоне от 10 000 до 1 000 000. Для тестового канала рассматривались доли подписчиков: 0.1%, 1%, 5% и 10%.

Измерения проводились по среднему времени выполнения хранимой процедуры за 20 повторов после 5 прогревочных запусков. Между повторениями счётчики сбрасывались, а статистика обновлялась командой `ANALYZE`. Для сравнения использовались следующие стратегии b-tree индексации:

- без индексов;
- индекс по `subscriptions(channel_id)`;
- индекс по `subscriptions(channel_id, user_id)`;
- индекс по `users(notifications_enabled, id)`;
- частичный индекс по `users(id)` с условием `notifications_enabled = true`;
- комбинация индексов `subscriptions(channel_id)` и `users(notification_id)`.

Для наглядности приведены относительные гистограммы, где для каждого

размера данных время нормировано по максимальному значению (0..1).

4.2 Анализ результатов

Относительные результаты для доли подписчиков 0.1% представлены на рисунке 4.1.

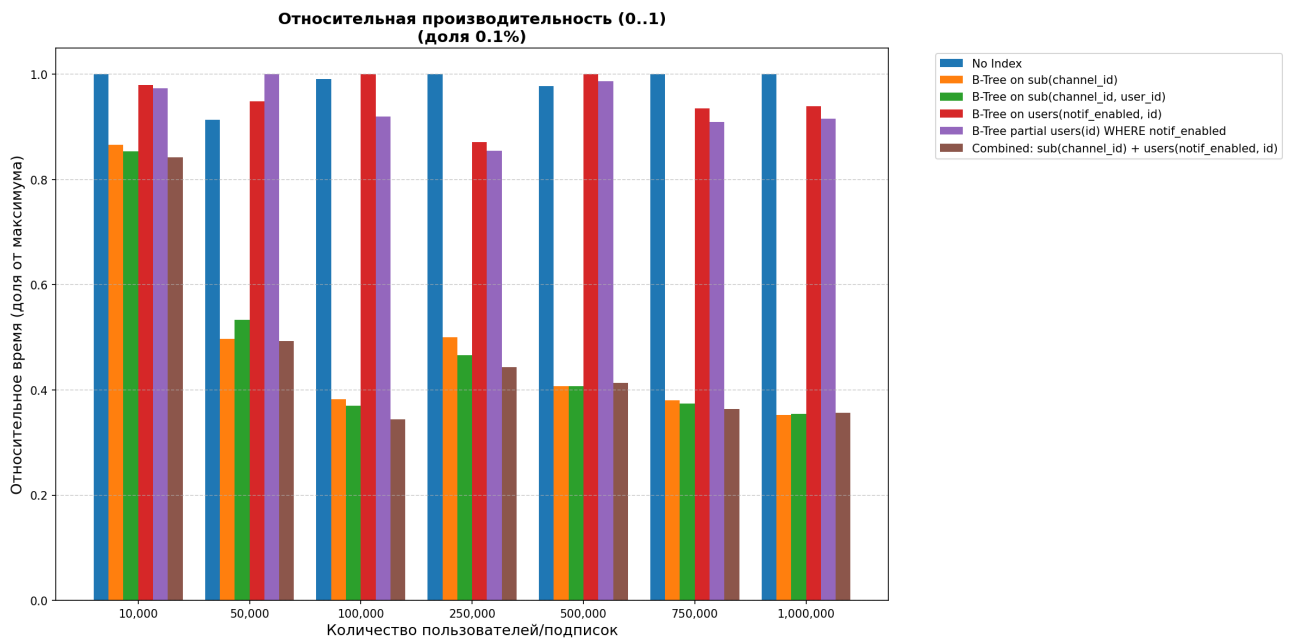


Рисунок 4.1 — Относительное время выполнения при доле подписчиков 0.1%

Для доли 1% результаты приведены на рисунке 4.2.

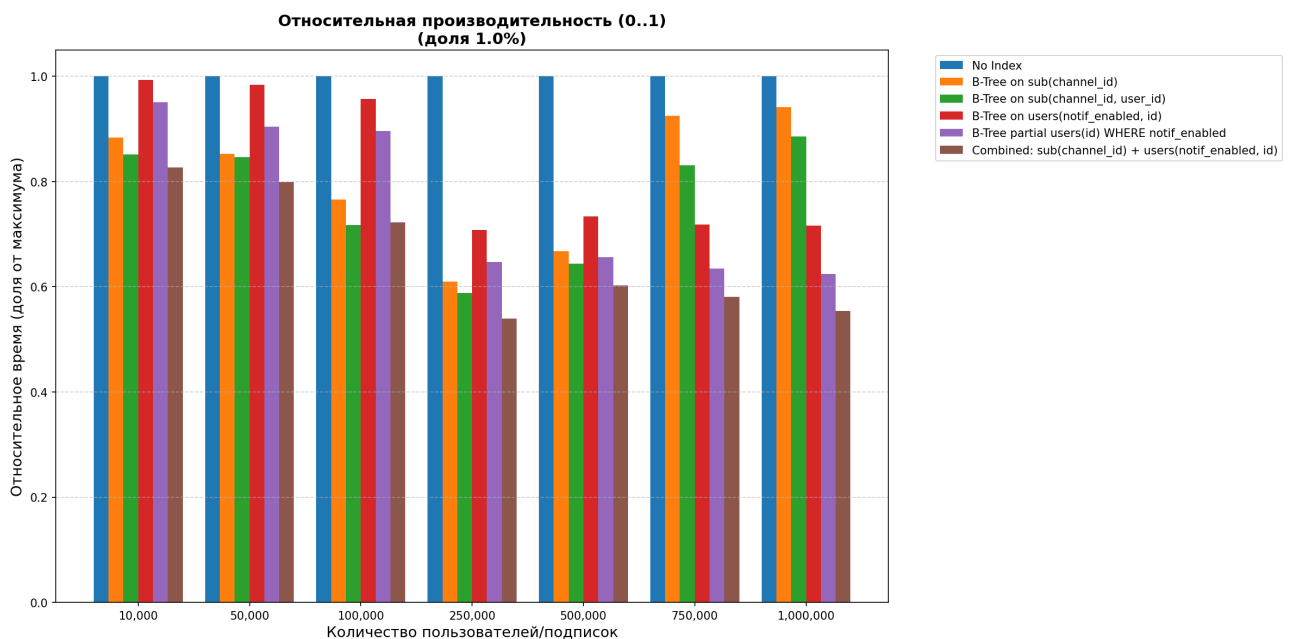


Рисунок 4.2 — Относительное время выполнения при доле подписчиков 1%

Для доли 5% результаты приведены на рисунке 4.3.

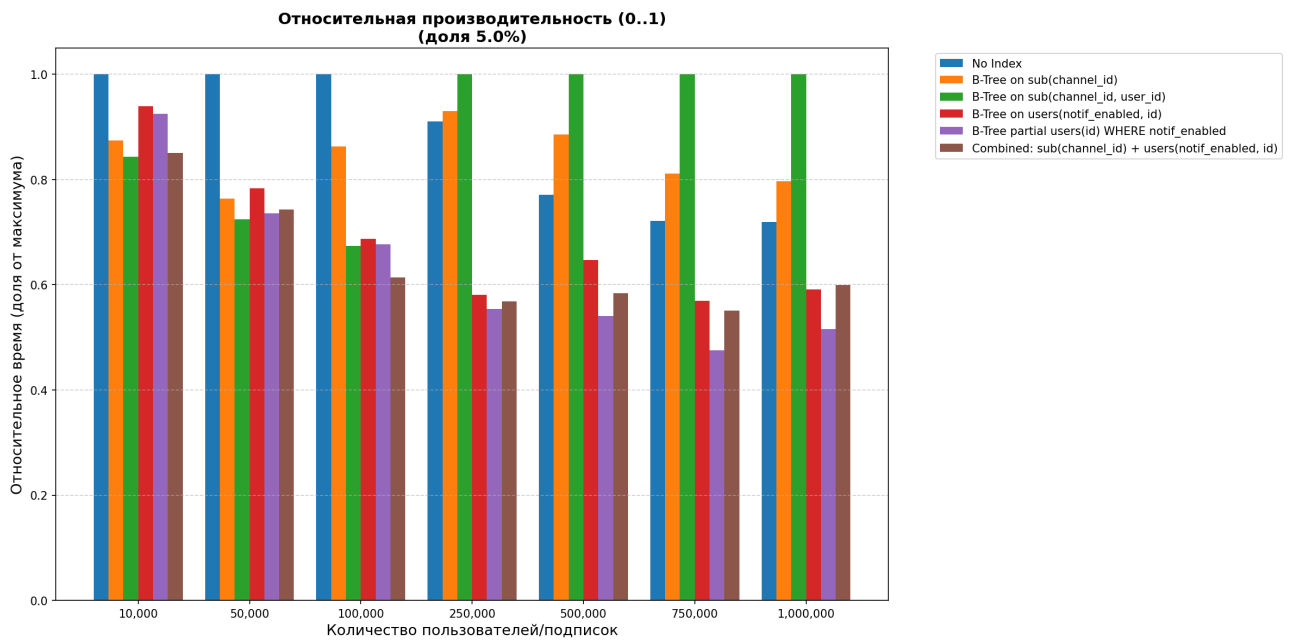


Рисунок 4.3 — Относительное время выполнения при доле подписчиков 5%

Для доли 10% результаты представлены на рисунке 4.4.

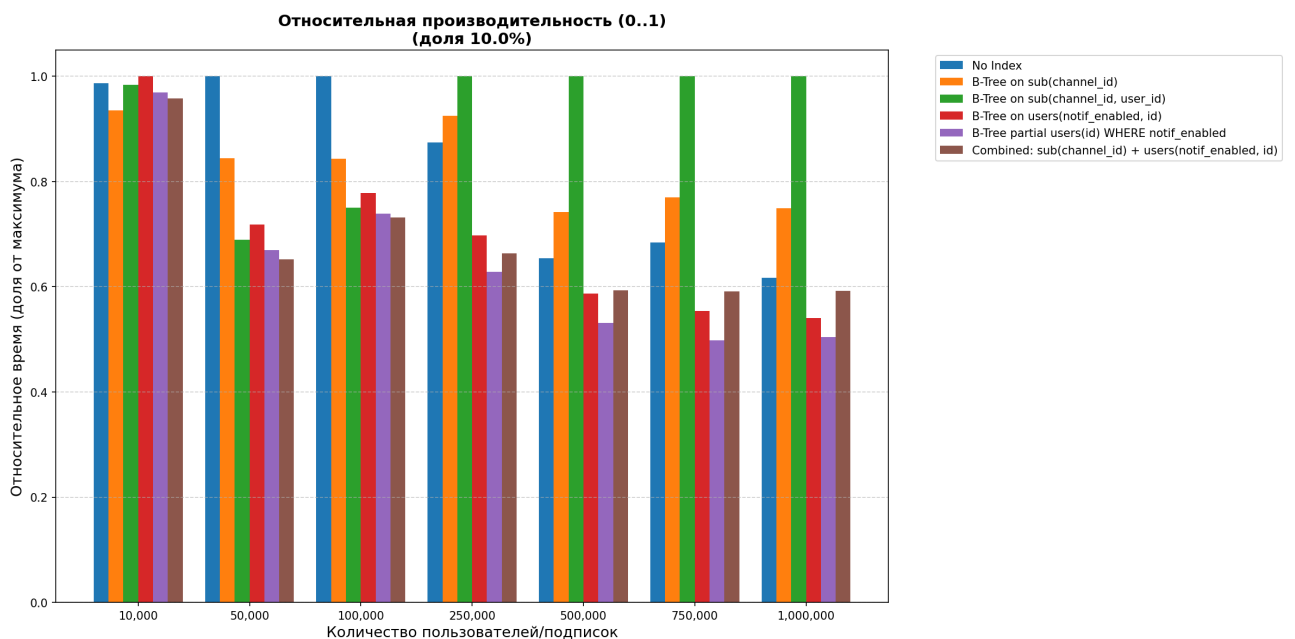


Рисунок 4.4 — Относительное время выполнения при доле подписчиков 10%

По результатам эксперимента видно, что чем меньше строк соответствует условию, тем сильнее индекс сокращает объём чтения, и тем больше выигрыша по времени.

Для рассматриваемой процедуры ключевым фильтром является атрибут `channel_id`. При малой доле подписчиков условие по идентификатору канала отбирает небольшой набор строк, поэтому индексы `subscriptions(channel_id)` и `subscriptions(channel_id, user_id)` дают заметное ускорение. По мере роста доли подписчиков ситуация меняется: диапазон ключей для `channel_id` становится большим, и индекс уже не даёт такого же сокращения объёма данных.

Частичный индекс `users(id)` с условием `notifications_enabled = true` не даёт яркого эффекта при малых долях, потому что основной объём работы уже решается фильтром по `channel_id`. При больших долях именно фильтрация по признаку уведомлений становится существенной, и частичный индекс даёт наибольший выигрыш.

Выводы по разделу

Таким образом, рекомендация для выбора стратегии индексирования зависит от доли подписчиков: для разреженных выборок лучше всего подходит индекс по `channel_id`, а для плотных — частичный индекс по признаку уведомлений. При этом второй вариант является более универсальным, так как он обеспечивает стабильный рост производительности независимо от доли подписчиков.

ЗАКЛЮЧЕНИЕ

Цель курсовой работы достигнута: разработана база данных для видеохостинг платформы и приложение, обеспечивающее доступ к ней.

Были решены все поставленные задачи:

- 1) проведён анализ предметной области;
- 2) выполнен анализ существующих решений и сформулированы требования к базе данных и приложению;
- 3) рассмотрены существующие модели хранения данных;
- 4) спроектированы база данных и ролевая модель на уровне СУБД;
- 5) разработано приложение, обеспечивающее доступ к базе данных;
- 6) описано тестирование разработанного программного обеспечения;
- 7) проведено исследование зависимости времени выполнения функции массового уведомления подписчиков канала о новом видео от количества подписчиков при различных стратегиях индексирования.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. VK. VK Видео [Электронный ресурс]. — Режим доступа: <https://vkvideo.ru> (дата обращения 19.03.2026).
2. RUTUBE. RUTUBE [Электронный ресурс]. — Режим доступа: <https://rutube.ru> (дата обращения 19.03.2026).
3. Дзен. Дзен Видео [Электронный ресурс]. — Режим доступа: <https://dzen.ru/video> (дата обращения 19.03.2026).
4. *Harrington J. L.* Relational database design and implementation: clearly explained. — 3-е изд. — Morgan Kaufmann, 2009. — 440 с. — ISBN 978-0123747303.
5. *Knuth D. E.* The art of computer programming. Т. 3. — 2-е изд. — Addison Wesley Longman, 1998. — 780 с. — ISBN 0-201-89685-0.
6. *Парфенов Ю. П.* Постреляционные хранилища данных. — Издательство Юрайт, 2025. — 97 с. — ISBN 978-5534211733.
7. *Andreas Meier M. K.* SQL & NoSQL Databases. — Springer Vieweg Wiesbaden, 2019. — 229 с. — ISBN 978-3-658-24548-1.
8. *Mesnier M., Ganger G. R., Riedel E.* Object-Based Storage // IEEE Communications Magazine. — 2003. — Т. 41, № 8. — С. 84—90. — DOI: 10.1109/MCOM.2003.1222722.
9. *Oracle Corporation.* MySQL 9.6 Reference Manual [Электронный ресурс]. — 2026. — Режим доступа: https://docs.oracle.com/cd/E17952_01/mysql-9.6-en/ (дата обращения 19.03.2026).
10. *The PostgreSQL Global Development Group.* PostgreSQL Documentation (Current) [Электронный ресурс]. — 2026. — Режим доступа: <https://www.postgresql.org/docs/current/> (дата обращения 19.03.2026).

11. *Oracle Corporation*. Oracle Database 21c Development [Электронный ресурс]. — 2026. — Режим доступа: <https://docs.oracle.com/en/database/oracle/oracle-database/21/development.html> (дата обращения 19.03.2026).
12. *Microsoft Corporation*. SQL Server Technical Documentation [Электронный ресурс]. — 2026. — Режим доступа: <https://learn.microsoft.com/en-us/sql/sql-server/> (дата обращения 19.03.2026).
13. *The SQLite Development Team*. Alphabetical List Of SQLite Documents [Электронный ресурс]. — 2026. — Режим доступа: <https://www.sqlite.org/docs.html> (дата обращения 19.03.2026).
14. *Amazon Web Services*. Amazon Simple Storage Service Documentation [Электронный ресурс]. — Режим доступа: <https://docs.aws.amazon.com/s3/> (дата обращения 19.03.2026).
15. *Google Cloud*. Cloud Storage Documentation [Электронный ресурс]. — Режим доступа: <https://cloud.google.com/storage/docs> (дата обращения 19.03.2026).
16. *MinIO, Inc.* MinIO Documentation [Электронный ресурс]. — Режим доступа: <https://min.io/docs/> (дата обращения 19.03.2026).
17. *The Go Team*. The Go Programming Language Documentation [Электронный ресурс]. — 2026. — Режим доступа: <https://go.dev/doc/> (дата обращения 24.05.2026).
18. *JetBrains*. GoLand Documentation [Электронный ресурс]. — 2026. — Режим доступа: <https://www.jetbrains.com/help/go/> (дата обращения 24.05.2026).

ПРИЛОЖЕНИЕ А

К расчётно-пояснительной записке курсовой работы прилагаются следующие материалы:

- 1) презентация к курсовой работе, состоящая из N слайдов;
- 2) USB-флеш-накопитель с исходными кодами, электронными версиями РПЗ и презентации.

ПРИЛОЖЕНИЕ Б

Листинг Б.1 — Сценарий создания базы данных

```
CREATE TABLE IF NOT EXISTS roles (  
    id BIGSERIAL PRIMARY KEY,  
    name VARCHAR(64) NOT NULL UNIQUE,  
    is_default BOOLEAN NOT NULL DEFAULT FALSE  
);  
  
CREATE TABLE IF NOT EXISTS users (  
    id BIGSERIAL PRIMARY KEY,  
    role_id BIGINT NOT NULL REFERENCES roles(id),  
    username VARCHAR(64) NOT NULL UNIQUE,  
    email VARCHAR(255) NOT NULL UNIQUE,  
    password_hash VARCHAR(255) NOT NULL,  
    notifications_enabled BOOLEAN NOT NULL DEFAULT TRUE,  
    is_active BOOLEAN NOT NULL DEFAULT TRUE,  
    created_at TIMESTAMP NOT NULL DEFAULT NOW(),  
    updated_at TIMESTAMP NOT NULL DEFAULT NOW()  
);  
  
CREATE TABLE IF NOT EXISTS channels (  
    id BIGSERIAL PRIMARY KEY,  
    user_id BIGINT NOT NULL REFERENCES users(id),  
    name VARCHAR(128) NOT NULL UNIQUE,  
    description TEXT,  
    created_at TIMESTAMP NOT NULL DEFAULT NOW()  
);  
  
CREATE TABLE IF NOT EXISTS videos (  
    id BIGSERIAL PRIMARY KEY,  
    channel_id BIGINT NOT NULL REFERENCES channels(id),  
    title VARCHAR(255) NOT NULL,  
    description TEXT,  
    filepath VARCHAR(512) NOT NULL,  
    created_at TIMESTAMP NOT NULL DEFAULT NOW()  
);  
  
CREATE TABLE IF NOT EXISTS playlists (  
    id BIGSERIAL PRIMARY KEY,
```

```

        channel_id BIGINT NOT NULL REFERENCES channels(id),
        name VARCHAR(255) NOT NULL,
        description TEXT,
        created_at TIMESTAMP NOT NULL DEFAULT NOW()
    );

CREATE TABLE IF NOT EXISTS playlist_items (
    playlist_id BIGINT NOT NULL REFERENCES playlists(id),
    video_id BIGINT NOT NULL REFERENCES videos(id),
    number INTEGER NOT NULL,
    added_at TIMESTAMP NOT NULL DEFAULT NOW(),
    PRIMARY KEY (playlist_id, video_id)
);

CREATE TABLE IF NOT EXISTS viewings (
    id BIGSERIAL PRIMARY KEY,
    user_id BIGINT NOT NULL REFERENCES users(id),
    video_id BIGINT NOT NULL REFERENCES videos(id),
    watched_at TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS comments (
    id BIGSERIAL PRIMARY KEY,
    user_id BIGINT NOT NULL REFERENCES users(id),
    video_id BIGINT NOT NULL REFERENCES videos(id),
    content TEXT NOT NULL,
    created_at TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS video_ratings (
    user_id BIGINT NOT NULL REFERENCES users(id),
    video_id BIGINT NOT NULL REFERENCES videos(id),
    liked BOOLEAN NOT NULL,
    rated_at TIMESTAMP NOT NULL DEFAULT NOW(),
    PRIMARY KEY (user_id, video_id)
);

CREATE TABLE IF NOT EXISTS comment_ratings (
    user_id BIGINT NOT NULL REFERENCES users(id),
    comment_id BIGINT NOT NULL REFERENCES comments(id),

```

```

        liked BOOLEAN NOT NULL,
        rated_at TIMESTAMP NOT NULL DEFAULT NOW(),
        PRIMARY KEY (user_id, comment_id)
    );

CREATE TABLE IF NOT EXISTS subscriptions (
    user_id BIGINT NOT NULL REFERENCES users(id),
    channel_id BIGINT NOT NULL REFERENCES channels(id),
    new_videos_count INTEGER NOT NULL DEFAULT 0,
    subscribed_at TIMESTAMP NOT NULL DEFAULT NOW(),
    PRIMARY KEY (user_id, channel_id)
);

```

Листинг Б.2 — Хранимая процедура для уведомления подписчиков канала о новом видео

```

CREATE OR REPLACE FUNCTION notify_subscribers_about_new_video(
    p_channel_id INT)
RETURNS VOID
LANGUAGE plpgsql
AS
$$
BEGIN
    UPDATE subscriptions s
    SET new_videos_count = s.new_videos_count + 1
    FROM users u
    WHERE s.channel_id = p_channel_id
        AND s.user_id = u.id
        AND u.notifications_enabled = TRUE;
END;
$$;

```